

POCKETSMART AI: YOUR SMART BUDGET

Naan Mudalvan Project Report

Sub Title: Validate Gemini API Connectivity

A structured 20-page project document covering API configuration, connectivity validation, testing, security, demonstration, documentation, and future scope.

Prepared for Academic Project Demonstration

1. Project Overview

PocketSmart AI: Your Smart Budget is a student-focused budgeting application designed to help users understand, record, and control their everyday financial activities. The application combines conventional budget tracking with an AI assistance layer powered through the Gemini API. The purpose of this project is to provide a simple digital companion that can answer budget-related questions, organize spending information, and provide understandable financial guidance.

Key Points

- Project title: PocketSmart AI: Your Smart Budget
- Module title: Validate Gemini API Connectivity
- Project category: AI-enabled personal budgeting application
- Primary goal: Simple, secure, and useful budget assistance
- Target users: Students, young adults, and everyday users
- Core technologies: Web application, backend services, database, Gemini API
- Major functions: Registration, login, budget entry, expense tracking, dashboard, AI assistance
- Expected benefit: Better awareness of income, expenses, and savings

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

2. Introduction to Gemini API Connectivity

Gemini API connectivity is the communication layer between PocketSmart AI and the selected Gemini model service. The application sends a carefully prepared request to the API and receives a response that can be displayed to the user. Connectivity validation confirms that the application can reach the service, authenticate correctly, submit a valid request, receive a response, and handle failures without crashing.

Key Points

- API means Application Programming Interface
- The API acts as a controlled communication bridge
- The application should keep credentials outside client-side code
- Requests should contain only the information required for the task
- Responses should be validated before being shown
- Timeouts and service errors should be handled gracefully
- Connectivity testing should be completed before full AI features are integrated

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

3. Problem Statement

Many students record expenses informally and may not have a clear picture of their spending pattern. A budgeting application can make the process easier, while an AI assistant can make financial information more understandable. However, AI features are useful only when the application has a reliable connection to the selected AI service. Therefore, this module focuses on validating Gemini API connectivity before depending on it for production features.

Key Points

- Users need simple budget information
- Expense entries should be organized consistently
- AI responses require reliable API communication
- Invalid credentials must be detected
- Network failures must not break the complete application
- Unexpected API responses must be handled safely
- The connectivity test should be repeatable during development

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

4. Objectives

The main objective is to verify that PocketSmart AI can communicate successfully with the Gemini API. The validation process should be small, controlled, measurable, and easy to troubleshoot. It should prove the communication path before the project uses AI responses in dashboards or recommendation features.

Key Points

- Configure the API access securely
- Create a minimal test request
- Send the request from the approved backend environment
- Receive and parse the API response
- Record success or failure status
- Test invalid or missing credentials safely
- Test network and timeout handling
- Avoid exposing secret keys in logs or user interfaces

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

5. System Architecture

A practical architecture separates the user interface, application server, AI integration layer, and data storage. The browser communicates with the application backend. The backend prepares an AI request and communicates with Gemini through the API integration layer. Keeping the API key on the server side reduces the chance of exposing the credential to users.

Key Points

- User Interface → Application Backend
- Application Backend → AI Service Connector
- AI Service Connector → Gemini API
- Gemini API → AI Service Connector
- AI Service Connector → Backend
- Backend → User Interface
- Database stores application data separately from API credentials

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

6. Requirements Analysis

The connectivity module requires a development environment capable of making HTTPS requests to the Gemini API. It also requires a valid API credential configured through a secure environment variable or secret-management mechanism. The application should have an identifiable health-check or test function so developers can validate the integration without depending on a complex user workflow.

Key Points

- Internet access during live connectivity testing
- Valid Gemini API credentials
- Backend runtime with HTTPS request support
- Environment-variable configuration
- JSON request/response handling
- Logging that excludes secrets
- Error-handling mechanism
- Test case documentation

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

7. API Configuration

API configuration should be centralized so that the project does not scatter credentials or endpoint settings throughout the source code. A common development approach is to store the credential in an environment variable and read it only on the server. Configuration files containing secrets should not be committed to a public repository.

Key Points

- Create the API credential using the approved provider workflow
- Store the credential in an environment variable
- Load configuration at application startup
- Check whether the credential exists
- Never print the credential in logs
- Use HTTPS for communication
- Keep model and endpoint configuration easy to update
- Document setup without publishing the secret

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

8. Connectivity Validation Workflow

The validation workflow begins by checking configuration, then constructing a minimal request. The backend sends the request to Gemini and waits for the response. The result is classified as success or failure. A successful response confirms the basic communication path, while a controlled error confirms that the application can recognize and report problems.

Key Points

- Step 1: Load configuration
- Step 2: Verify required settings exist
- Step 3: Build a minimal test prompt
- Step 4: Send an authenticated HTTPS request
- Step 5: Wait for the response
- Step 6: Validate response structure
- Step 7: Return a safe success/failure status
- Step 8: Record diagnostic information without secrets

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

9. Test Request Design

The connectivity request should be intentionally simple. The purpose is not to evaluate the full budgeting assistant, but to verify communication. A short prompt such as a request for a simple acknowledgement is sufficient for a connectivity test. The test should avoid sending real personal financial information.

Key Points

- Use a minimal prompt
- Avoid personal or sensitive user data
- Use the configured model correctly
- Set a reasonable request timeout
- Validate HTTP and API-level status information
- Validate that usable response content exists
- Keep the test independent of complex budget calculations

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

10. Response Validation

Receiving an HTTP response alone does not always prove that the integration is working correctly. The application should validate the response structure and identify whether the expected content is present. If the service returns an error object, the application should classify it as a failure and provide a safe diagnostic message to the developer.

Key Points

- Check transport-level success
- Check API-level error information
- Parse JSON only when the response format is valid
- Check expected response fields
- Reject empty or malformed AI content
- Avoid displaying raw internal errors to normal users
- Keep detailed diagnostics available to developers

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

11. Error Handling

A robust application anticipates authentication errors, invalid configuration, network failures, rate limits, timeouts, unavailable services, malformed responses, and unexpected server errors. Error handling should prevent a failed AI request from causing the entire budgeting application to stop working.

Key Points

- Missing API key → configuration error
- Invalid credential → authentication error
- Blocked request → permission/configuration error
- Rate limit → retry or user-friendly temporary message
- Timeout → controlled timeout response
- Network failure → connectivity message
- Malformed response → integration error
- Unexpected exception → safe fallback and developer log

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

12. Security Considerations

Security is essential because an API credential can provide access to a paid or rate-limited service. The project should treat the credential as confidential. Client-side JavaScript should not contain the secret. Logs, screenshots, source repositories, and documentation should not reveal the key.

Key Points

- Keep secrets on the server side
- Use environment variables or a secret manager
- Do not hard-code API keys
- Do not commit secret files
- Mask credentials in logs
- Use HTTPS
- Apply appropriate access controls
- Rotate exposed credentials when necessary
- Send only necessary application data

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

13. Functional Test Cases

The following test cases can be used to document validation. Each test checks a specific condition and records the expected behavior. Test results should be updated during actual implementation so that the report reflects the real application environment.

Key Points

- TC01 Valid credential + valid request → successful AI response
- TC02 Missing credential → clear configuration failure
- TC03 Invalid credential → authentication failure
- TC04 Network unavailable → controlled connectivity failure
- TC05 Request timeout → timeout handled safely
- TC06 Empty response → response validation failure
- TC07 API error response → error classified and logged safely
- TC08 Repeated valid request → stable connectivity behavior

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

Test ID	Condition	Expected Result
TC01	Valid configuration	Successful response
TC02	Missing key	Configuration error
TC03	Invalid key	Authentication error
TC04	Network failure	Controlled failure
TC05	Timeout	Timeout handled

14. Integration Testing

After the isolated API test succeeds, integration testing connects the AI module to the rest of PocketSmart AI. The objective is to confirm that the login state, budget data, backend routing, AI connector, and user interface work together without exposing secrets or breaking existing functionality.

Key Points

- Login and authorization are validated before protected operations
- Budget data is prepared in the expected format
- Backend route calls the AI connector
- AI connector handles the API response
- User interface receives a safe result
- AI failure does not erase saved budget information
- Logs contain useful diagnostics without credentials

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

15. Performance and Reliability

Connectivity should be reliable enough for normal user interactions. The project should measure response time during development and avoid unnecessary AI calls. Caching or request throttling may be considered where appropriate. The application should remain usable when the AI service is temporarily unavailable.

Key Points

- Measure request duration during testing
- Avoid repeated identical calls when unnecessary
- Use reasonable timeouts
- Handle rate limits gracefully
- Keep core budget tracking independent of AI availability
- Use retry logic carefully rather than retrying endlessly
- Monitor repeated integration failures

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

16. Sample Backend Logic

The implementation should use the official Gemini API documentation and the SDK or HTTP method appropriate to the chosen runtime. The conceptual flow is: load the secret, create the client, send a minimal request, inspect the result, and return a safe status. The exact endpoint and SDK syntax should match the currently supported Gemini API version used by the project.

Key Points

- loadSecret()
- if secret missing: return configuration error
- create Gemini client
- send minimal validation prompt
- receive response
- validate response structure
- return success status
- catch and classify expected errors
- never return the secret to the browser

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

17. Demonstration Procedure

For project demonstration, the developer can show the configured application, execute the connectivity test, display the successful response, and then explain how failures are handled. A demonstration should use test data rather than real personal financial information and should never display the API credential.

Key Points

- Open PocketSmart AI
- Navigate to the AI/API validation area
- Run the connectivity test
- Show successful response status
- Explain backend-to-Gemini communication
- Demonstrate safe error handling if appropriate
- Explain security controls
- Connect the validated module to later AI budgeting functions

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

18. Documentation and Maintenance

Good documentation makes the project easier to evaluate, maintain, and extend. The report should explain setup requirements, configuration, testing, known limitations, and troubleshooting steps. Documentation should describe where a secret is configured without publishing the secret itself.

Key Points

- Project purpose and scope
- System architecture
- Configuration instructions
- API connectivity workflow
- Test cases and expected results
- Error-handling approach
- Security practices
- Troubleshooting checklist
- Version and dependency notes
- Future maintenance plan

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

19. Results and Expected Outcome

Successful validation demonstrates that the PocketSmart AI backend can communicate with the Gemini service using the configured integration. The result becomes a foundation for future features such as budget explanations, spending summaries, savings suggestions, and conversational assistance. Actual performance and availability should be reported from measured project tests rather than assumed values.

Key Points

- Connectivity can be verified independently
- Configuration errors can be identified
- API responses can be validated
- Failures can be handled without crashing the application
- The AI module can be integrated into later features
- Security practices reduce accidental credential exposure
- Testing evidence supports project demonstration

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.

20. Conclusion and Future Scope

The Gemini API connectivity module is an important technical foundation for PocketSmart AI: Your Smart Budget. A controlled validation process proves the communication path before AI features are integrated into the main application. The project can later be extended with personalized budget summaries, category analysis, natural-language expense queries, savings planning, multilingual assistance, analytics, notifications, and improved monitoring. Future development should continue to prioritize privacy, security, reliability, and transparent handling of AI-generated information.

Key Points

- Final deliverable: validated AI connectivity workflow
- Next phase: integrate validated AI service with budget features
- Future enhancement: intelligent spending summaries
- Future enhancement: savings and goal assistance
- Future enhancement: multilingual user support
- Future enhancement: analytics and visualization
- Future enhancement: monitoring and automated testing
- Project principle: AI should assist users, while users remain in control of financial decisions

Implementation Note

During actual project implementation, replace generic validation statements with the exact SDK version, model name, endpoint, test date, measured response time, and observed test result. This keeps the report accurate and reproducible. Do not place API keys, passwords, or other secrets in the report.